Momentum based Gradient Descent

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambdaa=1):
    net = np.dot(X, weights)
    return 1 / (1 + np.exp(-lambdaa * net))

def calculate_dw(X, y, y_pred):
    return (y_pred - y) * y_pred * (1 - y_pred) * X

def calculate_error(y, y_pred):
    return np.square(y_pred - y)
```

```
def train_vanilla(X, y, weights, epochs=500, lr=0.001):
    for i in range(epochs):
        dw = np.zeros_like(weights)
        error = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        weights -= lr * dw
        error /= (2 * len(X))

        if (i + 1) % 100 == 1:
            print(f"Vanilla GD - Epoch {i+1}: Weights: {weights}, Error: {error}\n")
```

```
def train_sgd(X, y, weights, epochs=500, lr=0.001):
    for i in range(epochs):
        error = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw = calculate_dw(Xi, yi, y_pred)
            weights -= lr * dw
            error += calculate_error(yi, y_pred)

        error /= (2 * len(X))

        if (i + 1) % 100 == 1:
            print(f"SGD - Epoch {i+1}: Weights: {weights}, Error: {error}\n")
```

```
def train_minibatch(X, y, weights, epochs=500, lr=0.001, batch_size=32):
    for i in range(epochs):
        error = 0
        indices = np.random.permutation(len(X))
        X_shuffled, y_shuffled = X[indices], y[indices]

        for j in range(0, len(X), batch_size):
            X_batch = X_shuffled[j:j+batch_size]
            y_batch = y_shuffled[j:j+batch_size]
            dw = np.zeros_like(weights)

            for Xi, yi in zip(X_batch, y_batch):
                y_pred = predict(Xi, weights)
                dw += calculate_dw(Xi, yi, y_pred)
                error += calculate_error(yi, y_pred)

            weights -= lr * dw / batch_size

        error /= (2 * len(X))

        if (i + 1) % 100 == 1:
            print(f"Mini-Batch GD - Epoch {i+1}: Weights: {weights}, Error: {error}\n")
```

```
def train_momentum(X, y, weights, epochs=500, lr=0.001, gamma=0.9):
    velocity = np.zeros_like(weights)

    for i in range(epochs):
        dw = np.zeros_like(weights)
        error = 0
```

```
        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        velocity = (gamma * velocity) + (lr * dw)
        weights -= velocity
        error /= (2 * len(X))

        if (i + 1) % 100 == 1:
            print(f"Weights after {i+1} epochs: {weights}")
            print(f"Error after {i+1} epochs: {error}\n")
```

```
df = pd.read_csv('/content/bank_note.csv')
```

```
df.insert(4, 'x0', 1)
```

```
X = df[['variance', 'skewness', 'curtosis', 'entropy', 'x0']].values
y = df['class'].values
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=13, test_size=0.2)
```

```
weights = input("Enter 4 weights and 1 bias: ").split()
weights = np.array([float(weight) for weight in weights])
```

```
Enter 4 weights and 1 bias: 0.5 -0.3 0.8 -0.2 0.1
```

```
print("Training with Vanilla Gradient Descent:")
train_vanilla(X_train, y_train, weights.copy())
```

```
Training with Vanilla Gradient Descent:
Vanilla GD - Epoch 1: Weights: [ 0.40288631 -0.36975684  0.76737718 -0.21493266  0.10083191], Error: 0.22200047217131297

Vanilla GD - Epoch 101: Weights: [-1.15064665 -0.66846828 -0.74614346 -0.16946111  0.96252917], Error: 0.009030375982628718

Vanilla GD - Epoch 201: Weights: [-1.35427359 -0.7915461  -0.91217708 -0.12503201  1.37395896], Error: 0.0066003745603933186

Vanilla GD - Epoch 301: Weights: [-1.49476602 -0.87008135 -1.01762176 -0.10266398  1.61400397], Error: 0.005717041509239582

Vanilla GD - Epoch 401: Weights: [-1.60723228 -0.92984062 -1.09773271 -0.08984     1.77937919], Error: 0.005255149295901785
```

```
print("\nTraining with Stochastic Gradient Descent:")
train_sgd(X_train, y_train, weights.copy())
```

```
Training with Stochastic Gradient Descent:
SGD - Epoch 1: Weights: [ 0.40285907 -0.35152906  0.76387662 -0.22141326  0.10211463], Error: 0.21553256062269643

SGD - Epoch 101: Weights: [-1.14561893 -0.66476061 -0.74032973 -0.16417117  0.96683678], Error: 0.009029169444909996

SGD - Epoch 201: Weights: [-1.35154067 -0.78980504 -0.90826928 -0.12170598  1.37562248], Error: 0.006611536956683183

SGD - Epoch 301: Weights: [-1.49288638 -0.86914225 -1.01449405 -0.1000586   1.61497435], Error: 0.0057291249252229256

SGD - Epoch 401: Weights: [-1.6057342  -0.92931699 -1.09499752 -0.08757615  1.78017942], Error: 0.005267079756733141
```

```
print("\nTraining with Mini-Batch Gradient Descent:")
train_minibatch(X_train, y_train, weights.copy())
```

```
Training with Mini-Batch Gradient Descent:
Mini-Batch GD - Epoch 1: Weights: [ 0.49696757 -0.30215851  0.79897726 -0.20047278  0.10002777], Error: 0.22178741317262377

Mini-Batch GD - Epoch 101: Weights: [ 0.17252767 -0.37680188  0.65799813 -0.30952328  0.11449382], Error: 0.17809900100452813

Mini-Batch GD - Epoch 201: Weights: [-0.16288886 -0.32636623  0.3706844  -0.46240491  0.13788309], Error: 0.11172117496642026

Mini-Batch GD - Epoch 301: Weights: [-0.36417993 -0.3195185  -0.0271191  -0.46589504  0.1358979 ], Error: 0.05255153586724401

Mini-Batch GD - Epoch 401: Weights: [-0.48294149 -0.34985423 -0.24368494 -0.39179542  0.15081283], Error: 0.03129944200825642
```

```
print("Training with Momentum-Based Gradient Descent:")
train_momentum(X_train, y_train, weights.copy())
```

```
Training with Momentum-Based Gradient Descent:
Weights after 1 epochs: [ 0.40288631 -0.36975684  0.76737718 -0.21493266  0.10083191]
Error after 1 epochs: 0.22200047217131297
```

```
Weights after 101 epochs: [-4.24439425 -2.32929019 -2.84528457 -0.50129445  3.26867294]
Error after 101 epochs: 0.0038012737213713754

Weights after 201 epochs: [-4.17442757 -2.1573167  -2.72878316 -0.23825072  3.82535335]
Error after 201 epochs: 0.00339241374857784

Weights after 301 epochs: [-4.19280914 -2.14565401 -2.73179652 -0.18651172  3.98375096]
Error after 301 epochs: 0.0033659951927423482

Weights after 401 epochs: [-4.24011353 -2.16290861 -2.75935542 -0.1765494   4.05556288]
Error after 401 epochs: 0.0033581983984917797
```

Nesterov GD

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

def predict(X,weights,lambdaa = 1):
    net = np.dot(X,weights)
    return (1/(1 + np.exp(-lambdaa*net)))

def calculate_dw(X,y,y_pred):
    return ((y_pred - y)*(y_pred)*(1-y_pred)*X)

def calculate_error(yi,y_pred):
    return np.square(y_pred-yi)

def train_vanilla(X, y, weights, epochs=500, lr =0.001, gamma = 0.9):
    vel = 0
    for epoch in range(epochs):
        dw = 0
        error = 0
        w_lookahead = weights - (gamma*vel)
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,w_lookahead)
            dw += calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)

        vel = (gamma*vel) + (lr*dw)
        weights -= vel
        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_stochastic(X, y, weights, epochs=500, lr =0.001, gamma = 0.9):
    vel = 0
    for epoch in range(epochs):
        error = 0
        for Xi,yi in zip(X,y):
            w_lookahead = weights - (gamma*vel)
            y_pred = predict(Xi,w_lookahead)
            dw = calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)
            vel = (gamma*vel) + (lr*dw)
            weights -= vel

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_minibatch(X, y, weights, epochs=500, lr =0.001, gamma = 0.9, bs = 32):
    vel = 0
    for epoch in range(epochs):
        error = 0
        i = 0
        dw = 0
        w_lookahead = weights - (gamma*vel)
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,w_lookahead)
            dw += calculate_dw(Xi,yi,y_pred)
            i += 1
            error += calculate_error(yi,y_pred)
```

```
            if i%bs == 0 or i==len(X):
                vel = (gamma*vel) + (lr*dw)
                weights -= vel
                dw = 0
                w_lookahead = weights - (gamma*vel)

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


dataset = pd.read_csv('bank_note.csv')
dataset.insert(4,'x0',1)

X = dataset[['variance','skewness','curtosis','entropy','x0']].values
y = dataset['class']

X_train, X_test, y_train, y_test = train_test_split(X,y,random_state=13,test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')

print()
print('---- Nesterov Accelerator GD (Vanilla) ----')
train_vanilla(X_train,y_train,weights.copy())

print()
print('---- Nesterov Accelerator GD (Stochastic) ----')
train_stochastic(X_train,y_train,weights.copy())

print()
print('---- Nesterov Accelerator GD (Mini Batch) ----')
train_minibatch(X_train,y_train,weights.copy())
```

```
Enter 4 weights and 1 bias : 0.5 -0.2 0.8 -0.3 0.1

---- Nesterov Accelerator GD (Vanilla) ----
Weights after epoch 50 :  [-3.49117892 -2.04229559 -2.39552899 -0.60303988  2.31810234]
Error after epoch 50 :  0.00471687017130476396
Weights after epoch 100 :  [-3.45328151 -1.87224237 -2.30684568 -0.29353175  3.04883708]
Error after epoch 100 :  0.003640868839082657278
Weights after epoch 150 :  [-3.46815934 -1.82397708 -2.28487885 -0.18596515  3.2919615 ]
Error after epoch 150 :  0.003519926931212650565 9
Weights after epoch 200 :  [-3.50772965 -1.82738884 -2.30183423 -0.15340952  3.40904504]
Error after epoch 200 :  0.00349129173291124382 6
Weights after epoch 250 :  [-3.5580828  -1.84536887 -2.33049624 -0.1433506   3.48132738]
Error after epoch 250 :  0.003475212986003675605 5
Weights after epoch 300 :  [-3.61053969 -1.86769292 -2.36225834 -0.14131404  3.53603833]
Error after epoch 300 :  0.00346213123537973953 2
Weights after epoch 350 :  [-3.66203055 -1.89086716 -2.39412772 -0.1422255   3.58281365]
Error after epoch 350 :  0.003450582235017679323 3
Weights after epoch 400 :  [-3.71159094 -1.91365954 -2.42507957 -0.14419383  3.625303  ]
Error after epoch 400 :  0.00344020011540050373 2
Weights after epoch 450 :  [-3.7589919  -1.93566211 -2.45480636 -0.14650294  3.66500047]
Error after epoch 450 :  0.003430803212482962073 3
Weights after epoch 500 :  [-3.80426336 -1.95676882 -2.48326045 -0.1488785   3.70258114]
Error after epoch 500 :  0.003422259428265976622 3

---- Nesterov Accelerator GD (Stochastic) ----
Weights after epoch 50 :  [-1.71011165 -0.98706578 -1.15298314 -0.06378427  1.91758059]
Error after epoch 50 :  0.005036315826264799985 3
Weights after epoch 100 :  [-2.05882251 -1.15791344 -1.38076412 -0.05037447  2.29700764]
Error after epoch 100 :  0.00439618992864354755 6
Weights after epoch 150 :  [-2.30256689 -1.27164923 -1.53380356 -0.05229923  2.51406603]
Error after epoch 150 :  0.004143191222970830081 3
Weights after epoch 200 :  [-2.49411191 -1.3593854  -1.65252165 -0.05724172  2.67187902]
Error after epoch 200 :  0.003997635911901913690 4
Weights after epoch 250 :  [-2.65324811 -1.43183819 -1.75084608 -0.06275047  2.79912669]
Error after epoch 250 :  0.003899847897968876036 6
Weights after epoch 300 :  [-2.7899924  -1.49404024 -1.83539452 -0.06816443  2.90739901]
Error after epoch 300 :  0.003828346990995061832 8
Weights after epoch 350 :  [-2.91021985 -1.54879194 -1.90988946 -0.0732923   3.00248338]
Error after epoch 350 :  0.003773200545565705431 5
Weights after epoch 400 :  [-3.01770032 -1.59783178 -1.97665933 -0.07809039  3.08770966]
Error after epoch 400 :  0.003729074655666798640 3
Weights after epoch 450 :  [-3.11500845 -1.64232387 -2.03727072 -0.08256501  3.16519529]
Error after epoch 450 :  0.003692804009525691097 8
Weights after epoch 500 :  [-3.20398668 -1.68309186 -2.09283525 -0.08673895  3.23638781]
Error after epoch 500 :  0.003662370730114639818 4

---- Nesterov Accelerator GD (Mini Batch) ----
Weights after epoch 50 :  [-1.7168636  -0.98952896 -1.17209188 -0.08366107  1.91604304]
Error after epoch 50 :  0.005071936247684564511
Weights after epoch 100 :  [-2.05582376 -1.15546189 -1.39713639 -0.07460105  2.28819255]
Error after epoch 100 :  0.004454763939379085082
Weights after epoch 150 :  [-2.29437936 -1.26715218 -1.54859199 -0.07921971  2.49953739]
```

```
Error after epoch 150 :  0.0042073784443913837998
Weights after epoch 200 :  [-2.48215427 -1.35325202 -1.66573689 -0.08566432  2.65309761]
Error after epoch 200 :  0.00406416631035712701
Weights after epoch 250 :  [-2.63832689 -1.42424384 -1.76249298 -0.09202668  2.77691866]
Error after epoch 250 :  0.00396757790454604017 7
Weights after epoch 300 :  [-2.77264946 -1.48512929 -1.84552023 -0.09793734  2.88226802]
```

AdaGrad GD

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

def predict(X,weights,lambdaa = 1):
    net = np.dot(X,weights)
    return (1/(1 + np.exp(-lambdaa*net)))

def calculate_dw(X,y,y_pred):
    return ((y_pred - y)*(y_pred)*(1-y_pred)*X)

def calculate_error(yi,y_pred):
    return np.square(y_pred-yi)

def train_vanilla(X, y, weights, epochs=500, lr =0.01, epsilon = 1e-8):
    vel = 0
    for epoch in range(epochs):
        dw = 0
        error = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw += calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)

        vel = vel + (dw**2)
        weights -= ((lr*dw)/(np.sqrt(vel+epsilon)))
        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_stochastic(X, y, weights, epochs=500, lr =0.01, epsilon = 1e-8):
    vel = 0
    for epoch in range(epochs):
        error = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw = calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)
            vel = vel + (dw ** 2)
            weights -= ((lr * dw) / (np.sqrt(vel + epsilon)))

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_minibatch(X, y, weights, epochs=500, lr =0.01, epsilon = 1e-8, bs = 32):
    vel = 0
    for epoch in range(epochs):
        error = 0
        i = 0
        dw = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw += calculate_dw(Xi,yi,y_pred)
            i += 1
            error += calculate_error(yi,y_pred)

            if i%bs == 0 or i==len(X):
                vel = vel + (dw ** 2)
                weights -= ((lr * dw) / (np.sqrt(vel + epsilon)))
                dw = 0

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
dataset = pd.read_csv('bank_note.csv')
dataset.insert(4,'x0',1)

X = dataset[['variance','skewness','curtosis','entropy','x0']].values
y = dataset['class']

X_train, X_test, y_train, y_test = train_test_split(X,y,random_state=13,test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')

print()
print('---- Adagrad (Vanilla) ----')
train_vanilla(X_train,y_train,weights.copy())

print()
print('---- Adagrad (Stochastic) ----')
train_stochastic(X_train,y_train,weights.copy())

print()
print('---- Adagrad (Mini Batch) ----')
train_minibatch(X_train,y_train,weights.copy())
```

```
Enter 4 weights and 1 bias : 0.5 -0.3 0.8 -0.2 0.1

---- Adagrad (Vanilla) ----
Weights after epoch 50 :  [ 0.36918472 -0.39656945  0.6597786  -0.3537846   0.27041179]
Error after epoch 50 :  0.19567605886444735713
Weights after epoch 100 :  [ 0.3058142  -0.40740499  0.5871799  -0.43244154  0.35215603]
Error after epoch 100 :  0.17989405938334955516
Weights after epoch 150 :  [ 0.25652662 -0.40370433  0.52821245 -0.49053043  0.41031586]
Error after epoch 150 :  0.16592541038125384228
Weights after epoch 200 :  [ 0.21613909 -0.4069373   0.47822073 -0.53404856  0.45251852]
Error after epoch 200 :  0.1542208210789953718
Weights after epoch 250 :  [ 0.18205302 -0.41730777  0.43529201 -0.56783744  0.48468036]
Error after epoch 250 :  0.144555349793081099
Weights after epoch 300 :  [ 0.15250625 -0.42994374  0.39774515 -0.59516786  0.51058269]
Error after epoch 300 :  0.13646215064399017892
Weights after epoch 350 :  [ 0.12634786 -0.44241387  0.36429732 -0.61784617  0.53214554]
Error after epoch 350 :  0.1295823751968506434
Weights after epoch 400 :  [ 0.102809   -0.45390832  0.33403708 -0.63693576  0.55039595]
Error after epoch 400 :  0.12364962929621052336
Weights after epoch 450 :  [ 0.08135295 -0.46420935  0.30631278 -0.65313336  0.56595387]
Error after epoch 450 :  0.1184641672765370513
Weights after epoch 500 :  [ 0.06158895 -0.47329695  0.28064803 -0.66692978  0.5792263 ]
Error after epoch 500 :  0.113874966866007759426

---- Adagrad (Stochastic) ----
Weights after epoch 50 :  [-0.76485056 -0.43446906 -0.43632756 -0.22194463  0.54500453]
Error after epoch 50 :  0.016284535574453748115
Weights after epoch 100 :  [-0.90468521 -0.48984462 -0.53441307 -0.15136513  0.81095469]
Error after epoch 100 :  0.012050310321363587237
Weights after epoch 150 :  [-0.98383968 -0.52751251 -0.59483421 -0.11434777  0.98185739]
Error after epoch 150 :  0.010237048459835997128
Weights after epoch 200 :  [-1.04066238 -0.5566425  -0.63909609 -0.09044565  1.10569322]
Error after epoch 200 :  0.009195023858824663021 4
Weights after epoch 250 :  [-1.08579105 -0.58056735 -0.67425519 -0.07345177  1.20166061]
Error after epoch 250 :  0.008507855484644829915
Weights after epoch 300 :  [-1.1236148  -0.600952   -0.70354039 -0.0606521   1.27932502]
Error after epoch 300 :  0.008015688210736432273
Weights after epoch 350 :  [-1.15637663 -0.61875999 -0.72870544 -0.05063001  1.34412199]
Error after epoch 350 :  0.007643036397146483129 5
Weights after epoch 400 :  [-1.18538846 -0.63460213 -0.75081182 -0.04256032  1.39942357]
Error after epoch 400 :  0.00734933916410075292
Weights after epoch 450 :  [-1.21149139 -0.6488912  -0.77055238 -0.03592426  1.44745815]
Error after epoch 450 :  0.007110751821512124366
Weights after epoch 500 :  [-1.23526121 -0.6619199  -0.78840522 -0.03037664  1.48977067]
Error after epoch 500 :  0.006912289683919827472 4

---- Adagrad (Mini Batch) ----
Weights after epoch 50 :  [-0.23273155 -0.41478014 -0.08406427 -0.5987801   0.33025107]
Error after epoch 50 :  0.05790725174422044254
Weights after epoch 100 :  [-0.43989314 -0.40070166 -0.29772348 -0.41583776  0.36783079]
Error after epoch 100 :  0.029589629096072631927
Weights after epoch 150 :  [-0.55588285 -0.41955157 -0.37549239 -0.30917473  0.51161247]
Error after epoch 150 :  0.021498904904992891 85
Weights after epoch 200 :  [-0.63500597 -0.4370875  -0.42196287 -0.24666328  0.64054472]
Error after epoch 200 :  0.017637174023587068057
Weights after epoch 250 :  [-0.69460135 -0.45259098 -0.45716729 -0.20446898  0.74619648]
Error after epoch 250 :  0.015329300451961457192
Weights after epoch 300 :  [-0.74238878 -0.4667176  -0.48619249 -0.17363421  0.83407126]
```

ADAM GD

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

def predict(X,weights,lambdaa = 1):
    net = np.dot(X,weights)
    return (1/(1 + np.exp(-lambdaa*net)))

def calculate_dw(X,y,y_pred):
    return ((y_pred - y)*(y_pred)*(1-y_pred)*X)

def calculate_error(yi,y_pred):
    return np.square(y_pred-yi)

def train_vanilla(X, y, weights, epochs=500, lr =0.01, epsilon = 1e-8, beta1 = 0.9, beta2 = 0.999):
    mom = 0
    vel = 0
    for epoch in range(epochs):
        dw = 0
        error = 0

        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw += calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)

        mom = (beta1 * mom) + ((1 - beta1) * dw)
        vel = (beta2 * vel) + ((1 - beta2) * (dw ** 2))
        step = epoch + 1
        momcap = mom / (1 - (beta1 ** step))
        velcap = vel / (1 - (beta2 ** step))
        weights -= ((lr*momcap)/(np.sqrt(velcap+epsilon)))
        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_stochastic(X, y, weights, epochs=500, lr =0.01, epsilon = 1e-8, beta1 = 0.9, beta2 = 0.999):
    vel = 0
    mom = 0
    step = 0
    for epoch in range(epochs):
        error = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw = calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)
            mom = (beta1 * mom) + ((1 - beta1) * dw)
            vel = (beta2 * vel) + ((1 - beta2) * (dw ** 2))
            step += 1
            momcap = mom / (1 - (beta1 ** step))
            velcap = vel / (1 - (beta2 ** step))
            weights -= ((lr * momcap) / (np.sqrt(velcap + epsilon)))

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_minibatch(X, y, weights, epochs=500, lr =0.01, epsilon = 1e-8, beta1 = 0.9, beta2 = 0.999, bs = 32):
    vel = 0
    mom = 0
    step = 0
    for epoch in range(epochs):
        error = 0
        i = 0
        dw = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw += calculate_dw(Xi,yi,y_pred)
            i += 1
            error += calculate_error(yi,y_pred)

            if i%bs == 0 or i==len(X):
                mom = (beta1 * mom) + ((1 - beta1) * dw)
                vel = (beta2 * vel) + ((1 - beta2) * (dw ** 2))
                step += 1
                momcap = mom / (1 - (beta1 ** step))
```

```
                velcap = vel / (1 - (beta2 ** step))
                weights -= ((lr * momcap) / (np.sqrt(velcap + epsilon)))
                dw = 0

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


dataset = pd.read_csv('bank_note.csv')
dataset.insert(4,'x0',1)

X = dataset[['variance','skewness','curtosis','entropy','x0']].values
y = dataset['class']

X_train, X_test, y_train, y_test = train_test_split(X,y,random_state=13,test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')

print()
print('---- Adam (Vanilla) ----')
train_vanilla(X_train,y_train,weights.copy())

print()
print('---- Adam (Stochastic) ----')
train_stochastic(X_train,y_train,weights.copy())

print()
print('---- Adam (Mini Batch) ----')
train_minibatch(X_train,y_train,weights.copy())
```

```
Enter 4 weights and 1 bias : 0.5 -0.3 0.8 -0.2 0.1

---- Adam (Vanilla) ----
Weights after epoch 50 :  [-0.00864925 -0.46946873  0.23147369 -0.71486743  0.59612891]
Error after epoch 50 :  0.10494104630881491144
Weights after epoch 100 :  [-0.38399021 -0.49478318 -0.29626449 -0.6561284   0.49597997]
Error after epoch 100 :  0.0383051540003665537
Weights after epoch 150 :  [-0.63604616 -0.51831655 -0.49548006 -0.33021121  0.75173833]
Error after epoch 150 :  0.017033851169472355493
Weights after epoch 200 :  [-0.78482597 -0.53842935 -0.56983013 -0.17230139  1.08003421]
Error after epoch 200 :  0.011729694894199740119
Weights after epoch 250 :  [-0.89038566 -0.56666868 -0.62966877 -0.09494448  1.28885176]
Error after epoch 250 :  0.00961010034654348536
Weights after epoch 300 :  [-0.97526032 -0.59914243 -0.68248071 -0.05477388  1.4393417 ]
Error after epoch 300 :  0.0084416865273178751195
Weights after epoch 350 :  [-1.04765489 -0.63235322 -0.73051397 -0.0320891   1.55445804]
Error after epoch 350 :  0.007679567485198126654 7
Weights after epoch 400 :  [-1.11168359 -0.66469907 -0.77455114 -0.01864705  1.64668977]
Error after epoch 400 :  0.007134521881365031539
Weights after epoch 450 :  [-1.16968489 -0.69560975 -0.8152097  -0.01053579  1.72310067]
Error after epoch 450 :  0.006721637785334703153
Weights after epoch 500 :  [-1.22311928 -0.72495648 -0.85301208 -0.00572048  1.78806041]
Error after epoch 500 :  0.006396093916180978010 2

---- Adam (Stochastic) ----
Weights after epoch 50 :  [-4.63049307 -2.32944096 -2.9766457   0.14002867  4.51309329]
Error after epoch 50 :  0.0041016126439045821603
Weights after epoch 100 :  [-6.01599873 -3.02526523 -3.89188302  0.17020894  5.72948709]
Error after epoch 100 :  0.003908044306108407368
Weights after epoch 150 :  [-7.02373566 -3.54348879 -4.56218577  0.16382902  6.67836188]
Error after epoch 150 :  0.0038313970598111926789
Weights after epoch 200 :  [-7.87704804 -3.98585937 -5.13229781  0.11107911  7.42168874]
Error after epoch 200 :  0.0038311691776470678823
Weights after epoch 250 :  [-8.65000438 -4.38744698 -5.64579952  0.05046224  8.06808959]
Error after epoch 250 :  0.0038401955647165181956
Weights after epoch 300 :  [-9.36245731 -4.74328802 -6.10048646  0.01935936  8.71257741]
Error after epoch 300 :  0.003806185530109232813
Weights after epoch 350 :  [-10.32133414  -5.00599583  -6.35491979   0.01721076   9.74246747]
Error after epoch 350 :  0.003762150896044963727
Weights after epoch 400 :  [-11.33677061  -5.25596044  -6.61917726  -0.12058123   9.97536409]
Error after epoch 400 :  0.004216673603430927585
Weights after epoch 450 :  [-11.83442407  -5.50982801  -6.98318981  -0.09353233  10.88496668]
Error after epoch 450 :  0.004116485599836882945 13
Weights after epoch 500 :  [-11.9402351   -5.80419541  -7.5437901   -0.08840171  11.03357545]
Error after epoch 500 :  0.003846440829162655195 5

---- Adam (Mini Batch) ----
Weights after epoch 50 :  [-1.88018767 -1.04578128 -1.26867772 -0.00902356  2.31659937]
Error after epoch 50 :  0.0047495229474876509835
Weights after epoch 100 :  [-2.50354248 -1.33611091 -1.65605512 -0.04238876  2.75554822]
Error after epoch 100 :  0.004181616068786320618
Weights after epoch 150 :  [-2.92878508 -1.53035237 -1.91933317 -0.06600311  3.06188087]
Error after epoch 150 :  0.003966377794159776370 3
Weights after epoch 200 :  [-3.23952983 -1.67511904 -2.11467682 -0.08498061  3.29487717]
```

```
Error after epoch 200 :  0.003851702821472967 4345
Weights after epoch 250 :  [-3.4840186  -1.78991685 -2.26932672 -0.10006216  3.48277949]
Error after epoch 250 :  0.0037813039058408189548
Weights after epoch 300 :  [-3.68651829 -1.88523796 -2.39773873 -0.11227213  3.64105223]
```

RMSPROP

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

def predict(X,weights,lambdaa = 1):
    net = np.dot(X,weights)
    return (1/(1 + np.exp(-lambdaa*net)))

def calculate_dw(X,y,y_pred):
    return ((y_pred - y)*(y_pred)*(1-y_pred)*X)

def calculate_error(yi,y_pred):
    return np.square(y_pred-yi)

def train_vanilla(X, y, weights, epochs=500, lr =0.001, epsilon = 1e-8, beta = 0.5):
    vel = 0
    for epoch in range(epochs):
        dw = 0
        error = 0

        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw += calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)

        vel = (beta * vel) + ((1 - beta) * (dw ** 2))
        weights -= ((lr*dw)/(np.sqrt(vel+epsilon)))
        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_stochastic(X, y, weights, epochs=500, lr =0.001, epsilon = 1e-8, beta = 0.5):
    vel = 0
    for epoch in range(epochs):
        error = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw = calculate_dw(Xi,yi,y_pred)
            error += calculate_error(yi,y_pred)
            vel = (beta * vel) + ((1 - beta) * (dw ** 2))
            weights -= ((lr * dw) / (np.sqrt(vel + epsilon)))

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)


def train_minibatch(X, y, weights, epochs=500, lr =0.001, epsilon = 1e-8, beta = 0.5, bs = 32):
    vel = 0
    for epoch in range(epochs):
        error = 0
        i = 0
        dw = 0
        for Xi,yi in zip(X,y):
            y_pred = predict(Xi,weights)
            dw += calculate_dw(Xi,yi,y_pred)
            i += 1
            error += calculate_error(yi,y_pred)

            if i%bs == 0 or i==len(X):
                vel = (beta * vel) + ((1 - beta) * (dw ** 2))
                weights -= ((lr * dw) / (np.sqrt(vel + epsilon)))
                dw = 0

        error /= (2*len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
dataset = pd.read_csv('bank_note.csv')
dataset.insert(4,'x0',1)

X = dataset[['variance','skewness','curtosis','entropy','x0']].values
y = dataset['class']

X_train, X_test, y_train, y_test = train_test_split(X,y,random_state=13,test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')

print()
print('---- RMSProp (Vanilla) ----')
train_vanilla(X_train,y_train,weights.copy())

print()
print('---- RMSProp (Stochastic) ----')
train_stochastic(X_train,y_train,weights.copy())

print()
print('---- RMSProp (Mini Batch) ----')
train_minibatch(X_train,y_train,weights.copy())
```

```
Enter 4 weights and 1 bias : 0.5 -0.3 0.8 -0.2 0.1

---- RMSProp (Vanilla) ----
Weights after epoch 50 :  [ 0.44927722 -0.35026931  0.74911461 -0.25121534  0.15201945]
Error after epoch 50 :  0.21232381351993832818
Weights after epoch 100 :  [ 0.39924891 -0.39956105  0.69895294 -0.30156264  0.20255065]
Error after epoch 100 :  0.20263579025661369758
Weights after epoch 150 :  [ 0.3492193  -0.44706321  0.64881001 -0.35181577  0.25288175]
Error after epoch 150 :  0.19287119074663296093
Weights after epoch 200 :  [ 0.29908513 -0.40435338  0.5985697  -0.40206719  0.30306762]
Error after epoch 200 :  0.18198364410774883452
Weights after epoch 250 :  [ 0.24902834 -0.3848629   0.54837732 -0.45213804  0.35307667]
Error after epoch 250 :  0.16943068372710684684
Weights after epoch 300 :  [ 0.19905202 -0.39321154  0.49825867 -0.50209804  0.40302518]
Error after epoch 300 :  0.15629270595061976284
Weights after epoch 350 :  [ 0.1491182  -0.41179128  0.44819149 -0.5519769   0.45291541]
Error after epoch 350 :  0.14331460659000898549
Weights after epoch 400 :  [ 0.09921199 -0.436441    0.39816636 -0.60178291  0.50275551]
Error after epoch 400 :  0.13101140328307537847
Weights after epoch 450 :  [ 0.04932564 -0.46505528  0.34817237 -0.65151046  0.55254423]
Error after epoch 450 :  0.11970765465147113914
Weights after epoch 500 :  [-5.46470084e-04 -4.96191260e-01  2.98198109e-01 -7.01139218e-01
  6.02267007e-01]
Error after epoch 500 :  0.10955600267530620967 4

---- RMSProp (Stochastic) ----
Weights after epoch 50 :  [-2.87324887 -1.28259077 -1.76434183  0.38582114  4.41397171]
Error after epoch 50 :  0.005155501391850589126 4
Weights after epoch 100 :  [-3.77174825 -1.72845341 -2.38591983  0.49935458  5.68265357]
Error after epoch 100 :  0.005209315843811696737 7
Weights after epoch 150 :  [-4.53542924 -2.09060766 -2.86255744  0.54579096  6.49106923]
Error after epoch 150 :  0.005039394026772008660 25
Weights after epoch 200 :  [-5.23127054 -2.411847   -3.28460705  0.57684804  7.12872466]
Error after epoch 200 :  0.004837854186605542255 8
Weights after epoch 250 :  [-5.93516597 -2.675037   -3.67129021  0.62852967  7.66765583]
Error after epoch 250 :  0.004610514943513334885 4
Weights after epoch 300 :  [-6.64670768 -2.90294404 -4.03289711  0.71220381  8.15826461]
Error after epoch 300 :  0.004403244394019662241
Weights after epoch 350 :  [-7.35213693 -3.10526315 -4.37850664  0.80772799  8.61968433]
Error after epoch 350 :  0.004289568634655697963
Weights after epoch 400 :  [-8.02066214 -3.2980438  -4.7097415   0.89434006  9.0655461 ]
Error after epoch 400 :  0.004289460789639664704
Weights after epoch 450 :  [-8.64593276 -3.49351359 -5.03141234  0.95881591  9.49241929]
Error after epoch 450 :  0.004334239538423736314
Weights after epoch 500 :  [-9.2481555  -3.70571762 -5.35698037  1.00042743  9.90149893]
Error after epoch 500 :  0.004364740924503310157

---- RMSProp (Mini Batch) ----
Weights after epoch 50 :  [-0.9116205  -0.48617408 -0.51112031 -0.21787078  0.66515804]
Error after epoch 50 :  0.01360958247609864343
Weights after epoch 100 :  [-1.342404   -0.70570929 -0.83806715 -0.052544    1.49461421]
Error after epoch 100 :  0.006616594242533306555
Weights after epoch 150 :  [-1.62002167e+00 -8.46800220e-01 -1.03133859e+00 -3.03026335e-04
  1.89235283e+00]
Error after epoch 150 :  0.005302298366312973871 4
Weights after epoch 200 :  [-1.83433691 -0.94845704 -1.16967212  0.01513679  2.14326108]
Error after epoch 200 :  0.004779921223708300115 8
Weights after epoch 250 :  [-2.01395997 -1.03444053 -1.28436398  0.01689638  2.32866441]
```